KVRouter 实施步骤详解（白话版）

> 这份文档把《05-实施指南-P2》第 14 节那张 11 步的表格逐条展开，讲清楚每一步在做什么、为什么要做它、做到什么程度算过关。原文档里的设计细节（那些 § 编号）全部保留对应关系，随时可以翻回去看完整设计。写作原则和 P1 白话版一样：尽量说人话，每个术语第一次出现时顺手解释。P2 是两个大项目里技术密度更高的一个，所以这份文档比 P1 那份写得更细。

先弄清楚：这个项目在做什么，跟 P1 什么关系

项目代号 KVRouter，全称 KV-Cache 感知推理网关。先用人话把电梯陈述翻译一遍：

大模型推理服务通常会同时跑多个引擎副本（可以理解成多个”分店”），网关站在用户和这些分店中间，决定每个请求发给谁。引擎内部有个东西叫 KV Cache（已算过的中间结果缓存）：如果两个请求的开头一模一样（比如都带同一个很长的系统提示词），第二个请求就能直接复用第一个请求算好的缓存，首字响应快一大截。所以网关越了解”每个副本手里有什么缓存”，路由就越聪明。

这个项目的三条主线，对应你的三个身份：

1. **复现者**：vLLM 引擎（社区最流行的大模型推理引擎）从 v0.10 开始会主动推送缓存事件，告诉你”我缓存了哪些块、淘汰了哪些块”。你要把依赖这种事件的”精准模式”和只靠路由历史猜的”估算模式”各亲手实现一遍。
2. **测量者**：你的实现要和真实部署的 AIBrix（vllm-project 官方的推理基础设施项目，本项目的头号对手）在同一套负载下同台跑分。厂商只发布”精准模式更准”的结论，“到底准多少、什么负载下差距会消失”的曲线由你来发布。
3. **前沿贡献者**：弹性伸缩期的缓存冷启动协同（新副本缓存全空、热身慢、甚至被饿死）是上游没解决好的问题。你先测量它、再设计解法、最后回馈给 AIBrix。

与 P1 的时间关系：P2 的现勘（步骤 0）轻量部分**现在就并行启动**，每周投入不超过半天，不抢 P1 的主线时间；P1 收尾之后再全职转 P2。

文档用法和 P1 一样：原指南 §5 到 §10 按”零件”组织（引擎摸底、反代基座、负载台、双模式索引、弹性协同、增量池），§14 那张表按”时间”组织。你以 §14 的步骤顺序为主线，每接到一步就回去读对应设计章节。原指南还给出了一条拼接逻辑，值得背下来：**步骤 1+2 得到一个可测量的基座；3+4+5 构成双模式复现（第一个完整故事）；6+7 构成前沿纵深（第二个故事，也是 PR 的来源）；9 用真值收口；10 把信用落到上游**。你在任意一层停下来都能写简历，但秋招包的目标是完成 1–7 加 10。

名词小词典

名词	大白话解释
vLLM	社区最流行的大模型推理引擎，本项目扮演”被路由的引擎”。
副本（replica）	同时运行的一个 vLLM 实例。多个副本一起扛流量。

名词	大白话解释
网关 / 路由器	挡在用户和一堆副本中间的程序，决定每个请求发给哪个副本。
prefill / decode	推理的两个阶段。prefill 把整段输入算一遍、吐出第一个字（计算密集）；decode 之后一个字一个字往外蹦（访存密集）。
TTFT	Time To First Token，从发出请求到收到第一个字的延迟，本项目的头号指标。
TPOT / TBT	相邻两个输出字之间的平均间隔 / 相邻两批的时间。
KV Cache	引擎把已算过的注意力中间结果存下来，省得每生成一个字都重算整段输入。
前缀命中（prefix hit）	新请求的开头和已缓存的前缀一样时，直接复用缓存，跳过 prefill 的重算。
KV 事件	vLLM v0.10 起引擎主动推送的消息：“我新增缓存了哪些块 / 淘汰了哪些块”。
块（block）/ block hash	引擎按约 256 个 token 一块来管理缓存；块的键由递归哈希链算出。
递归哈希	h_i = H(h_{i-1} ‖ token_i)：每多一个 token，就拿上一个哈希接着算，天然形成前缀链。
精准模式	网关消费引擎 KV 事件，实时知道”谁手里有什么块”。
估算模式	网关只靠自己送出去过的请求历史来猜，与引擎解耦。已被行业淘汰，但必须亲手复现。
radix 树 / trie	前缀树：相同前缀的键共享同一条树枝，天然适合查”最长公共前缀”。
倒排 / 正排索引	块 → 持有它的副本们（路由查询用）；副本 → 它持有的块们（摘除副本时清理用）。
tokenize / tokenizer	文本和 token 序列之间的翻译器。网关要把请求映射到引擎的块哈希，必须先拿到 token 序列。
SSE	Server-Sent Events，流式输出的 HTTP 协议，token 逐个推给客户端。
AIBrix	vllm-project 下的推理基础设施项目，头号对标对手，最终回馈目标。
llm-d / Production Stack	另外两个同类开源方案，现勘时要跑通或读码。
Envoy	业界主流的代理/网关底座，AIBrix 数据面基于它，我们不重造。
helm	K8s 的包管理器，一条命令部署一整套组件。

名词	大白话解释
kind	在自己电脑上用 Docker 模拟的迷你 K8s 集群。
CPU 后端	vLLM 可以不用 GPU、纯 CPU 跑小模型（如 Qwen2.5-0.5B，5 亿参数），开发期够用。
Prometheus / Grafana	监控数据的抓取存储系统 / 配套的图表看板。
/metrics	服务暴露监控数据的 HTTP 地址，Prometheus 定时来抓。
loadgen（负载台）	生成测试流量的程序。本项目里它是”一级公民”，所有实验数据的唯一出口。
泊松到达	一种随机但速率稳定的请求到达模式，排队论的标准假设。
Zipf 分布	少数键特别热、多数键很冷的分布，真实 prompt 池就长这样。
EWMA	指数加权移动平均，给抖动的数字做平滑。
Little’s Law	排队论公式：系统里平均请求数 = 到达率 × 平均耗时。用它算”需要几个副本”。
HPA	K8s 的自动扩缩容器，按 CPU 等指标自动加减副本。
饿死	新副本因为没缓存而接不到流量，永远热不起来。
drain	缩容前让副本先”排干”：停接新请求，等在途请求跑完再杀。
幂等	同一个请求执行两次和执行一次效果一样，重试才安全。
CountSketch	一种省内存的频率估计结构，回答”这个块大概被用过多少次”。
mock	用假数据顶替真依赖。CPU 阶段拿不到真 KV 事件时，用符合格式约定的假事件流开发逻辑。
LMCache	现成的 KV 卸载组件，把 KV 分层存到内存/SSD。只做集成实验，不自建。
P/D 分离	把 prefill 和 decode 跑在不同机器上，各自选最合适的硬件。只调研不实现。

步骤 0：现勘——2 到 3 周，闸门制，轻量部分现在就启动

设计细节见：§4

这一步的目标：摸清两件事——vLLM 的事件机制到底长什么样（格式、字段、推送通道），AIBrix 的路由实现到底做到什么程度。摸清之后用证据选定核心 2 要钻的缺口。这是整个项目的地基，也是开源贡献指南阶段 0 在 P2 侧的交付物。

时序说明：现勘的轻量部分（跑通环境 + 读码）**现在就并行启动**，每周不超过半天，不抢 P1 主线；重结论部分（风险关闭 + 缺口选定）在 P1 收尾前完成即可。

具体做什么：

1. **引擎摸底**：用 vLLM 的 CPU 后端跑通多副本——kind 集群里起 3 个副本，本机再起 2 个，模型用 Qwen2.5-0.5B-Instruct（5 亿参数，CPU 跑得动）。通读启动参数中与 KV 池（比如 `VLLM_CPU_KVCACHE_SPACE`）、prefix caching、KV 事件上报有关的选项，逐个做笔记。
2. **对手部署**：AIBrix 用官方 quickstart（helm 一键部署）成功即可，**不要求在本地编译整个全家桶**；另外把 Production Stack 的 helm 和 prefix-aware 教程也跑通。
3. **读码 + PR 笔记**：精读 AIBrix router 源码的四个部分——软评分、prefix index、KV 事件消费、伸缩相关；再精读 **10 个最近合入的 PR**，每个都做结构化笔记：diff 长什么形状、解决了什么问题、reviewer 关注什么、测试是怎么写的。这份笔记同时是开源贡献的预习材料。
4. **关闭头号技术风险（现勘第一周必须完成，不许带着假设开工）**：vLLM 的 CPU 后端会不会上报与 GPU 后端相同格式的 KV 事件？
   - 支持的话：精准模式全程在本地开发，省下大量租卡钱；
   - 不支持的话：CPU 阶段的精准模式只能用”符合锁定格式的 mock 事件流”开发逻辑，所有性能真值都要靠租 GPU 获得，预算和排期据此重算。
5. **用缺口闸门定下核心 2**（证据说了算，不能靠许愿）：

候选缺口	判定它成立需要看到的证据	若被上游提前解决
A（默认押注）伸缩期缓存冷启动：冷副本饿死、热身慢、缩容损失没有度量	在 AIBrix 上人为制造一次扩容事件，新副本在 5 分钟内拿到的流量份额和命中率仍然显著偏低；官方没有预热或放量机制	转做 B 或 C
B 精准与估算的差距曲线	官方没有公开”共享比例 × 欠配度”的二维对比	仍然可以做，但升不了核心 2
C 流式故障安全重试	mid-stream 时 kill 副本会错误重放或挂死，issue 区无人认领	A 被解决时它升级为核心 2

6. **产出《P2 对手现勘报告》**：部署命令与版本锁定（镜像 digest）、各系统的 feature 矩阵、复现到的官方 benchmark 数据、带着证据的缺口排序表。

过关标准：现勘报告评审通过，才能进入步骤 1。

步骤 1：基座——流式反代 + 指标层 + 负载台，三件一起搭

设计细节见：§5.1、§5.2、§5.3

这一步的目标：搭出可测量的地基。做完这一步，你手里就有了”能稳定转发流式请求、能看清所有指标、能按参数生成任意负载”的实验台。后面每一步的效果全靠它呈现。

具体做什么：

1. **SSE 流式反代**（这是你自研数据面的全部范围，难点全在流式）。SSE 是大模型流式输出的标准协议：token 一个一个以事件形式推给客户端。Go 标准库的 `httputil.ReverseProxy` 能当反代，但默认行为对 SSE 非常不友好，**五个坑你必须逐条实测，之后写进博客**：

- **坑 1**：反向代理默认会缓冲数据，你必须把 `FlushInterval` 设为 -1，它才肯逐 token 下发；
- **坑 2**：客户端断开时，你必须取消（cancel）上游的 context，否则引擎会继续烧算力，生成一堆没有人消费的 token；
- **坑 3**：SSE 禁止压缩。如果 `Accept-Encoding` 处理不当，gzip 的缓冲会把流式输出攒成”一次性爆发”，用户看到的就成了一卡一卡的；
- **坑 4**：空闲超时（idle timeout）是随着每个事件重置的，它按请求总时长来算——理解错了会误杀长请求；
- **坑 5**：TTFT 的准确定义是”从请求发出到收到第一个内容型 SSE 事件（`data:` 开头且不是心跳帧）的时间”，它既不是首字节时间，也不是收到 HTTP 头的时间。口径错了，后面所有实验数字全错。

配套工程点：读写超时分离（`ReadHeaderTimeout` 有值，响应流不设总超时）；每个副本一个连接池；加 `/health` 探针，连续失败就把副本摘除；`X-Request-Id` 全链路透传——步骤 8 的安全重试和步骤 7 的 drain 都会复用它。

2. **MetricsView 抽象先行**：vLLM `/metrics` 的指标名会随版本漂移（TTFT/TPOT 直方图、prefix cache 命中数、running/waiting 请求数、缓存占用率）。业务代码只能依赖你自己定义的抽象接口；针对锁定版本写一份字段映射，并处理好缺字段时的降级。指标名散落在业务代码里，就等于监控系统第一课直接挂掉。所有指标都带三个标签：`mode`（rr/estimator/precise/aibrix）、`replica`、`experiment_id`；实验配置（模型、vLLM 版本、镜像 digest、负载参数）随报告一起落盘。
3. **负载台**（一级公民，所有旗舰图的唯一出口）。它是一个参数化的场景生成器，参数固定写在实验配置文件里，保证三种自研策略与 AIBrix 对照组吃到**完全相同的流量**。参数分六个维度：
   - **共享前缀**：共享比例 f 取 0 / 0.25 / 0.5 / 0.8 四档；system prompt 池 100 个，热度服从 Zipf 分布（少数特别热）；
   - **对话形态**：单轮短问 / 多轮（轮数服从几何分布，均值 4 轮）/ 长文档问答（2KB 固定前缀加各不相同的问题）；
   - **到达过程**：泊松到达，速率 λ 扫点；突增实验用 3 倍 λ 持续 60 秒；
   - **token 形状**：输入/输出长度分布参数化，一切以锁定 tokenizer 的实际切分为准；
   - **缓存压力**：调整 KV 池大小，制造充裕、临界、欠配三档；
   - **副本增减**：脚本按时间表执行扩容 +1 / 缩容 -1，供事件研究使用。

   负载台输出：TTFT/TBT 直方图、吞吐、各副本在途请求数与 KV 占用、前缀命中率（引擎自报）、新副本拿到的流量份额。

   这里有一个陷阱：客户端自己先饱和了，这批数据就作废。所以你要常驻监控 generator 的 CPU 与连接数，最终结论只在”服务端饱和、客户端不满载”的区间里成立。
4. **Grafana 看板**：把上面所有指标摆上去，实验时肉眼可查。

过关标准：流式对话稳定；杀掉后端不会拖垮连接；负载台可以切换三种策略。

步骤 2：基线报告——先把最笨的策略比出高下

这一步的目标：用两种最简单的路由策略做第一份对照报告，同时标定负载台的工作区。最笨的策略是所有后续策略的”零点”，零点不稳，后面全歪。

具体做什么：

1. **两种基线策略**：RR（round-robin，轮流分发，完全不看缓存）和 least-outgoing（发给在途请求最少的副本，只看负载不看缓存）；
2. 固定负载，各跑至少 7 次取中位数，报告 TTFT、吞吐、各副本在途请求数的分布；
3. **标定工作区**：扫一遍 λ，找出”服务端已经开始排队、客户端还没饱和”的区间，画出来；后面所有实验都只在这个区间里做。

过关标准：对照报告产出；**客户端不饱和的证据随报告附上**。

步骤 3：估算索引——亲手复现一条已经被淘汰的路线

设计细节见：§6.1

这一步的目标：实现估算模式——网关只靠自己送出去过的请求历史，猜”哪个副本可能有什么前缀”。行业已经淘汰了这条路线，可别小看这个作业：你只有亲手做一遍，才能把”它为什么会被精准模式取代”讲透，这是上好的面试教学点。

具体做什么：

1. **前缀键的取法**：把对话消息按条取哈希，得到序列 `[h(system), h(user1), h(asst1), ...]`；radix 树（前缀树）的键就是这个哈希序列的前缀。按消息边界切、按哈希比较，避免存整段文本。
2. **数据结构**：树节点带三个东西——子节点表、`replicas`（每个副本最近一次见到该前缀的 unix 秒）、读写锁。
3. **信息源与生命周期**：唯一的信息源是网关自己的路由历史；TTL 默认 10 分钟，每次命中就刷新；副本的 Pod UID 发生变化或健康状态摆动时，把对应条目全量摘除；索引自身用 LRU 限制内存。
4. **诚实标注它的误差**：它只会朝一个方向出错——“不同文本恰好有相同 token 前缀”时会误判成命中（保守方向的误差）；但它完全感知不到引擎内部的淘汰，TTL 本质上是在猜。

过关标准：机制正确；文档里明确标注”对已淘汰路线的复现”。

步骤 4：精准索引——消费引擎事件，项目的技术核心

设计细节见：§6.2

这一步的目标：实现精准模式——直接消费 vLLM 推送的 KV 事件，实时维护”哪个副本手里有哪些块”的索引。这是两个索引里更难、也更有价值的一个。

具体做什么：

1. **tokenize 选型（最关键的取舍点）**：想把请求内容映射到引擎的块哈希，网关必须先拿到 token 序列。按顺序实验三个方案：
   - **方案 1（默认选它）**：调用引擎的 `/tokenize` 端点。零新依赖，代价是多一跳网络往返；对 token 序列做 LRU 缓存（同一个 prompt 池的命中率会很高）；先实测这一跳在路由决策耗时里占多大比例；
   - **方案 2**：部署一个 Python tokenizer 边车（sidecar，跟网关一起跑的辅助容器）。省下一跳，但引入新组件，还带来”模型词汇表怎么同步”的问题；
   - **方案 3**：在 Go 里内嵌 tokenizer。生态太弱，不选。

- 决策规则：数据证明那一跳不可接受时才升级到方案 2——**这个”用数据驱动选型”的过程本身就是上好的面试素材**。

2. **块标识对齐 AIBrix 的公开口径**：递归哈希 `h_i = H(h_{i-1} || token_i)`，按约 256 token 一块聚合成块哈希；请求的前缀命中深度 = 它的块哈希链与索引匹配的最长块数 × blockSize。**事件的具体格式（字段名、消息类型、推送通道）一律以锁定版本的 vLLM 源码或官方文档为准，现勘报告里必须包含一份格式摘录和对应版本号**——任何文档（包括本指南）都不能替你保证字段名。
3. **双向索引**：`block2rp`（块 → 持有它的副本集合，路由查询走它）+ `rp2blocks`（副本 → 它持有的块集合，摘除副本时可以一次清干净）+ `freq`（CountSketch 记录块热度，步骤 7 预热选 top-K 时用）。
4. **事件处理分三类**：新增（add）、淘汰（evict）、全量重建。**从收到事件到索引可见之间的延迟要单独打点**——索引滞后正是精准模式的主要误差来源，这个数字步骤 5 分析时要用。
5. **失配自愈**：当你把请求路由到”按理应该有前缀”的副本，引擎却自报未命中（原因可能是事件丢失、索引滞后或淘汰竞争），这属于性能退化，不算错误。按副本监控未命中率，一旦超过阈值，就触发该副本索引的全量重同步。
6. **路由打分**：`score = α·affinity + (1−α)·balance`，affinity 是命中块数的归一化值，balance = 1 − 在途请求数/容量。注意：α 的扫描实验已经降级为教学实验，它不是旗舰卖点。

过关标准：索引滞后有打点；事件格式锁定版本；失配重同步有测试。

步骤 5：旗舰图 1——三方对照，第一个故事收口

设计细节见：§ 6.3

这一步的目标：回答两个问题——精准模式到底比估算模式准多少？这个差距随”共享前缀比例 × 缓存欠配程度”怎么变？这是项目第一个完整故事。

具体做什么：

1. 跑全矩阵实验：三种条件（自研估算 / 自研精准 / 真实 AIBrix）× 共享前缀比例 f 四档 × 缓存压力三档，全部由同一个负载台供流；
2. 每组至少 7 次取中位数，附带 bootstrap 95% 置信区间；
3. 画二维曲线：横轴 f，纵轴 TTFT P50/P99 与前缀命中率，三条线画在同一张图上；
4. **结论允许指向任何方向**——包括”f < 0.2 时三者没有差异，为精准模式承担与引擎耦合的代价并不值得”。阴性边界比阳性数字更值钱，因为它回答了”这一招在什么时候没有用”。

过关标准：二维曲线加 95% CI；阴性边界写清楚。

步骤 6：冷启动测量——用事件研究把痛点钉死

设计细节见：§ 7.1

这一步的目标：把”伸缩期的缓存协同没人做好”从一句话变成三组可量化的数字。**这批数字就是你未来给 AIBrix 提 issue 的全部弹药**，所以测量必须严谨到能见人。

事件研究（event study）是什么：以某个事件发生的时刻为原点（这里是新副本 Ready），在它前后的时间窗里观察各项指标怎么变化。金融和市场研究里常用这招，用在系统测量上同样合适。

具体做什么：

1. 在自研精准路由和真实 AIBrix 上**各做一轮**：以 t0 = 新副本 Ready 为原点，观察窗口从 −60 秒到 +300 秒；
2. 画四条时间轴：全局 TTFT P50/P99、全局命中率、新副本的流量份额、新副本自己的命中率；
3. 从中量化两个量：
   - **冷启动惩罚**：新副本达到稳态流量份额所需要的时间。饿死的判定标准：5 分钟内份额低于公平份额的 50%；
   - **扩容收益迟滞**：从扩容决策发生，到全局 P99 实际改善之间的滞后；
4. **缩容侧**：在当前常见策略（按在途请求数或副本年龄决定杀谁）下，记录每次被杀掉的副本中**独有块所占的比例**——这就是缓存价值损失，独有块被杀就真没了。

过关标准：饿死、迟滞、价值损失三组量化齐备，可直接作为 issue 素材。

步骤 7：协同设计——项目灵魂三件套 + 签名图

设计细节见：§7.2、§7.3、§7.4

这一步的目标：针对步骤 6 量化的三个痛点，完整实现三个协同机制，并产出全项目的签名图。这是 P2 技术含量最高的一步。

具体做什么：

1. **扩容前的热前缀预热**：
   - 从块热度统计里取出 top-K 热块，反推出覆盖这些块所需的最短前缀集合（贪心集合覆盖，K 与预算都参数化）；
   - 向新副本发送**合成的 prefill 请求**：长前缀加 `max_tokens=1`——只买 prefill、不买 decode，花最少的算力把 KV 灌进去；
   - 给预热设 token 预算上限（比如 KV 池的 15%）和速率限制，防止预热抢走在线请求的算力；
   - **预热进度要靠块事件来确认**，不能假设请求发出去就生效；未确认的块不计入新副本的就绪度。
2. **加权放量（带迟滞，不要一刀切换流量）**：新副本就绪之后，不立刻让它拿满分流量。权重 `w = clamp((readiness − θlo) / (θhi − θlo), 0, 1)`，其中 readiness = 已确认预热块 / 目标块；w 作为乘子进入软评分，权重在 θlo/θhi 之间线性爬坡；如果观察窗口（默认 30 秒）内份额波动超过阈值，就冻结爬坡防止抖动。老副本的 w 恒为 1。
3. **缓存价值感知缩容 + drain**：
   - 选谁杀：副本价值 = 它所有块的稀缺度之和，稀缺度 = 1 ÷ 持有该块的副本数——多家共有的块杀掉也没损失，独有块最值钱。优先杀价值低的副本，价值并列再比在途请求数；
   - drain 流程：先给副本打标记、停止接收新会话；然后等在途的流式请求结束（默认 120 秒，超时之后只对可以安全重试的请求强切）；等待期间该副本的块进入”濒死”状态，亲和打分降权但归零；
   - 防护线：任何时刻，可服务副本数都不得低于按 Little’s Law 算出的需求下限；某次缩容会突破这条线，就立即中止。
4. **Little’s Law 预测器（配角，不单独作为卖点）**：

- 到达率用 EWMA 平滑（α=0.3）；平均在系统请求数 `L = λ × (TTFT_prefill + 输出 token 数 × TPOT)`；需求副本数 `N = ceil(L ÷ 单副本容量)`，容量按”KV 占用 90% 时实测的稳态并发”标定；
- **它和冷启动故事的结合点**：预测的扩容提前量 = 预热耗时（实测值）+ 爬坡耗时，这样能做到”副本 Ready 的时候，热前缀也就绪了”；
- 自己写一个小控制器，直接调 Deployment 的 scale 接口（避开 prometheus-adapter 这个重依赖）；扩容立即生效，缩容设 5 分钟冷却；
- 保留三个对照组：基于 CPU 利用率的 HPA（开发期用它替代 GPU 利用率这个错误信号，报告里说明等价性论证）、基于 KV 占用的反应式策略、本策略。

5. **旗舰图 2（全项目签名图）**：让三种条件（反应式 HPA / 精准路由但无协同 / 完整协同）跑在同一个副本增减脚本下，对比它们的时间轴——TTFT 尖峰的高度与持续时长、命中率凹陷、扩容收益迟滞。

过关标准：旗舰图 2 产出；预热、放量、缩容三件套全部有测试；每个参数都有数据来源。

步骤 8：增量池——只挑一个做（默认 A：流式故障语义）

设计细节见：§8

这一步的目标：从三个候选增量里挑 **1 个**做完，选多了就是范围膨胀。默认选 A。

A 流式故障语义与安全重试（默认）：

- 用 `X-Request-Id` 做幂等键；
- 区分两类故障点：prefill 完成前副本挂掉——没有副作用，可以安全重试；prefill 完成后 mid-stream 挂掉——请求已经非幂等，要么靠客户端续传，要么显式重发，文档要把边界写清楚；
- 故障注入方式：在随机时刻 kill 副本；
- 产出一份独立的小报告。

B 多租户缓存干扰测量：透传租户标识，量化”一个租户的大批量流量驱逐了另一个租户的前缀”，软隔离手段是按租户做块价值保护与预热配额。什么时候选它：你对多租户叙事更感兴趣、或现勘发现 A 的边界文档其实很清楚时。

C TTFT 归因观测：把 TTFT 拆成排队 / prefill 重算（未命中）/ KV 命中 / 网络四段。工程很小但分析价值很高，是所有图的解释力底座。

过关标准：一份独立的小报告。

步骤 9：真值校准——租 GPU 换真数字

设计细节见：§10

这一步的目标：CPU 上跑出来的数字只对机制和相对趋势负责，绝对值没有生产意义。租一次 GPU，把两张旗舰图换成真硬件数字。

具体做什么：

1. 租 GPU（Qwen 7B 级别的模型）重跑旗舰图 1 和旗舰图 2；

2. 可选：用 LMCache 做一次 KV 卸载的集成实验（观察内存/SSD 分层对 TTFT 的影响）；选 2 张卡跑一次 P/D 分离的官方 demo，写一篇观感；
3. 用真数据替换 CPU 数据，**两个版本的数字都保留**，并在报告里说明差异。

过关标准：两张旗舰图都有 GPU 真值版本；CPU 版与 GPU 版差异解释清楚。

步骤 10：回馈上游——把故事讲给 AIBrix

设计细节见：§7.5

这一步的目标：把”我仓库里有个功能”升级成”上游社区认了我的判断”。你自己的仓库与上游 PR 属于同一条故事线的两个阶段，不要做成两个不相干的项目。

具体做什么：

1. 先在自己的仓库里把原型验证完整；
2. 向 AIBrix 提 issue，贴上步骤 6 的测量数据与复现脚本；
3. maintainer 认可方向之后，按照 AIBrix 的架构（Envoy 插件或 Go plugin）提交 routing policy 或预热控制器的 PR。

过关标准：至少 1 个代码级 PR（文档和测试的 PR 不计入）；即使被拒，也要保留”为什么被拒”的完整故事——它同样是面试素材。

调研层（只写文档或做集成实验，绝不自建）

- **P/D 分离与 KV 传输**：NIXL/SHFS/Mooncake、AIBrix 的 PrisKV、llm-d 的路线；租 GPU 的时候选 2 张卡跑一次官方 demo 写观感。你的口径永远是”我调研过，也理解为什么需要 RDMA 这类高速互连”。
- **KV offload**：用 LMCache 做一次集成实验就够了，不自建卸载层。
- **Registry 高可用**：行业答案已经出来（AIBrix 用 Redis 做状态同步）。本项目诚实地把范围限定为单实例加周期快照，另写一页决策分析——解释为什么第一天就不该上 etcd/Redis（写路径很热、读多写少、允许过期），但不去构建。

实验硬纪律（六条，违反任何一条数字就作废）

1. 每组实验至少跑 **7 次取中位数**，并报告 bootstrap 95% CI；只跑 3 次的数字不进报告；
2. **冷态控制**：每轮重启 vLLM 副本以清空 KV；热态实验要明确预热流程；轮次之间进程不得残留；
3. **完全锁定**：模型版本、vLLM 版本、镜像 digest、机器配置、负载配置文件全部随报告入库，做到 `make bench COND=xxx` 一键复现；
4. **三条件同流量**：自研两种模式与 AIBrix 共用同一份固定随机种子的请求轨迹文件；
5. **CPU 数据诚实标注**：CPU 模式只对机制与相对趋势负责；两张旗舰图最终都要由 GPU 真值替换，报告保留两个版本并说明差异；
6. **客户端哨兵**：负载台自己的 CPU 与连接数监控曲线作为每张性能图的附件；客户端接近饱和的轮次一律作废重跑。

排期与砍项顺序

- **立即开始（与 P1 并行）**：步骤 0 的轻量部分（vLLM CPU 多副本 + AIBrix quickstart + 读码），每周投入不超过 0.5 天，不抢 P1 的主线时间；
- **P1 收尾之后全职做 P2**：步骤 1–5 大约需要 9–11 周；6–7 大约 6–8 周；8 大约 2–3 周；9 大约 1 周；
- **2027 年 8 月秋招包的最低构成**：基座 + 核心 1（含旗舰图 1）+ 核心 2 主实验（旗舰图 2 初稿）+ 至少 1 个 AIBrix 代码级 PR（文档/测试 PR 不计入；如果 PR 流程还没闭合，至少要做到 issue 有 maintainer 的正式回应）；
- **砍项顺序（时间不够时）**：增量池 → GPU 校准里的 demo 类部分（旗舰图校准不能砍）→ 核心 2 的 drain 细化；**核心 1 与旗舰图 1 任何时候不砍**。